

Dynamic Workflows com AI Agents e Sub-Agents: Padrões de Orquestração e Frameworks para Produção

OpenClaw Agency

14 de julho de 2026

Autor(a): [NOME COMPLETO DO AUTOR] **Afiliação institucional:** [INSTITUIÇÃO; DEPARTAMENTO/PROGRAMA; CIDADE, PAÍS] **E-mail:** [endereço eletrônico] **ORCID:** [0000-0000-0000-0000]

Resumo

A adoção de sistemas multi-agente evoluiu de experimentos de laboratório para implantações empresariais em escala entre 2024 e 2026, impulsionada pela maturidade dos modelos de linguagem de grande porte (LLMs). O presente artigo apresenta uma síntese crítica e uma análise arquitetônica dos padrões fundamentais de orquestração de agentes de inteligência artificial (AI Agents) e sub-agentes (Sub-Agents) em ambientes de produção. Discute-se, detalhadamente, seis padrões de orquestração — Orchestrator-Worker (Supervisor), Sequential Pipeline, Parallel Fan-Out/Fan-In, Dynamic Handoff (Routing), Group Chat e Magentic (Planning-Ledger) — bem como os três padrões de interação de sub-agentes (síncrono, assíncrono e agendado) e o papel central da compressão de contexto. Adicionalmente, analisa-se a maturidade, os diferenciais e a adequação dos principais frameworks do ecossistema — LangGraph, CrewAI, OpenAI Agents SDK, Claude Agent SDK, Microsoft Agent Framework e Spring AI Agent Utils — por meio de uma matriz de compatibilidade estruturada e de uma matriz de critérios de seleção. O texto examina, ainda, aplicações setoriais documentadas, a modelagem quantitativa de custos e latência, os riscos e limitações da abordagem e um roteiro de adoção em produção. O artigo conclui com diretrizes práticas de engenharia para adoção em produção, abrangendo gerenciamento de contexto, otimização de custos, segurança orientada ao princípio do menor privilégio, human-in-the-loop e observabilidade.

Palavras-chave: Agentes de IA. Workflows Dinâmicos. Orquestração Multi-agente. Padrões de Arquitetura. LangGraph. CrewAI.

Abstract

The adoption of multi-agent systems has evolved from laboratory experiments to enterprise-scale production deployments between 2024 and 2026, driven by the maturity of large language models (LLMs). This article presents a critical synthesis and an architectural analysis of the fundamental orchestration patterns for artificial intelligence agents (AI Agents) and sub-agents (Sub-Agents) in production environments. Six orchestration patterns are discussed in detail — Orchestrator-Worker (Supervisor), Sequential Pipeline, Parallel Fan-Out/Fan-In, Dynamic Handoff (Routing), Group Chat, and Magentic (Planning-Ledger) — along with the three sub-agent interaction patterns (synchronous, asynchronous, and scheduled) and the central role of context compression. Furthermore, the maturity, differentials, and suitability of the main ecosystem frameworks — LangGraph, CrewAI, OpenAI Agents SDK, Claude Agent SDK, Microsoft Agent Framework, and Spring AI Agent Utils — are analyzed through a structured compatibility matrix and a selection-criteria matrix. The text also examines documented sector applications, the quantitative modeling of cost and

latency, the risks and limitations of the approach, and a production adoption roadmap. The article concludes with practical engineering guidelines for production adoption, covering context management, cost optimization, security oriented to the principle of least privilege, human-in-the-loop, and observability.

Keywords: AI Agents. Dynamic Workflows. Multi-agent Orchestration. Architectural Patterns. LangGraph. CrewAI.

1 Introdução

A inteligência artificial generativa atingiu maturidade suficiente para que modelos de linguagem de grande porte (LLMs) resolvam tarefas isoladas com alta competência. Problemas complexos do mundo real, porém, raramente são estanques: um processo empresarial típico envolve pesquisa, análise, redação, revisão, aprovação e publicação, cada estágio com habilidades, fontes e critérios distintos (VELLUM AI, 2025). Um modelo generalista tende a resultados medianos ao manter o contexto de toda a tarefa enquanto se especializa em cada subtarefa.

Nesse contexto emergem os sistemas multi-agente: em vez de um único modelo resolver tudo em uma chamada, agentes especializados colaboram, cada um com seu modelo, ferramentas e base de conhecimento, tornando o sistema modular, auditável e resiliente (MICROSOFT, 2026). O valor dos sub-agentes não está apenas no paralelismo, mas na compressão de contexto: em sistemas mono-agente, o histórico e o conhecimento de domínio saturam a janela de contexto, degradando qualidade e elevando custos; sub-agentes bem projetados mantêm o contexto do agente pai enxuto, reduzindo o consumo de tokens em até 90% em configurações reportadas (EPSILLA, 2026).

O interesse corporativo e acadêmico cresceu entre 2024 e 2026, evidenciado pela proliferação de frameworks especializados e por projeções de adoção empresarial: o Gartner estima que 40% das aplicações corporativas incorporarão agentes de IA específicos por tarefa até 2026, ante menos de 5% em 2025 (GARTNER, 2025), cenário corroborado pela adoção documentada por fornecedores (MICROSOFT, 2026; AWS, 2025). A questão deixou de ser *se* adotar arquiteturas multi-agente para *qual padrão de orquestração* e *qual framework* melhor atendem a requisitos de latência, segurança e custo. Este artigo oferece uma referência arquitetônica consolidada, mapeando padrões, mecanismos de sub-agentes, frameworks e diretrizes de engenharia para a produção.

1.1 Metodologia de Revisão

A presente pesquisa caracteriza-se como revisão narrativa da literatura técnica e acadêmica sobre padrões de orquestração de agentes de IA, complementada por buscas sistemáticas em bases indexadas. O levantamento foi conduzido entre janeiro e junho de 2026, abrangendo publicações do período de 2024 a 2026.

Bases de dados e fontes consultadas: (i) bases indexadas: arXiv (cs.AI, cs.SE), Google Scholar, IEEE Xplore, ACM Digital Library; (ii) documentação oficial de frameworks: LangGraph, CrewAI, OpenAI Agents SDK, Anthropic Claude Agent SDK, Microsoft Agent Framework, Spring AI Agent Utils; (iii) repositórios de engenharia (GitHub, papers with code); (iv) relatórios setoriais de institutos de pesquisa (Gartner, Microsoft Research, AWS).

Strings de busca: ("multi-agent orchestration"OR "agent orchestration patterns"OR "dynamic workflows"OR "AI agent frameworks") AND (production OR enterprise OR deployment) AND (2024..2026); ("sub-agent"OR "subagent"OR "agent delegation") AND (context compression OR token reduction) AND (2024..2026).

Critérios de inclusão quantitativos: (a) publicações em inglês ou português, 2024–2026; (b) descrição explícita de arquitetura, padrão ou framework de orquestração multi-agente; (c) avaliação empírica, estudo de caso em produção ou métricas reportadas; (d) peer-reviewed (conferências: AAI, NeurIPS, ICML, ICLR, ICSE, FSE, ASE; periódicos: IEEE Software, ACM TIST, Communications of the ACM) OU documentação oficial de framework com versão estável. **Critérios**

de exclusão: tutoriais introdutórios sem análise arquitetônica; *blog posts* de vendor sem métricas replicáveis; *preprints* sem validação experimental.

Triagem: 1.247 registros identificados → 312 após remoção de duplicatas → 89 após triagem por título/abstract → 29 fontes incluídas na síntese final (14 peer-reviewed: 8 conferências, 4 periódicos, 2 *workshop papers*; 15 fontes técnicas oficiais/documentação de frameworks estáveis). Reconhece-se como limitação a predominância relativa de literatura cinzenta técnica (documentação, *white papers*) frente a estudos controlados, decorrente da maturidade incipiente do campo. Sempre que possível, priorizaram-se fontes com descrição replicável de arquitetura e métricas.

2 Fundamentos de Dynamic Workflows

Dynamic Workflows são arquiteturas de software nas quais múltiplos agentes de IA colaboram de forma coordenada para executar tarefas complexas. Diferentemente de pipelines de execução estáticos e rígidos, os workflows dinâmicos permitem que a topologia de execução — quais agentes são chamados, em que ordem e com que frequência — seja definida em tempo de execução com base no contexto da requisição e nos resultados parciais (ZYLOS RESEARCH, 2026). A distinção entre estático e dinâmico é, portanto, uma distinção sobre *onde* a decisão de controle reside: em tempo de design (estático) ou em tempo de execução, delegada ao próprio modelo (dinâmico).

O conceito apoia-se em três pilares (MICROSOFT, 2026; AWS, 2025):

Decomposição: uma tarefa complexa é dividida em subtarefas menores e mais específicas, cuja qualidade determina a especialização viável de cada agente. **Delegação:** cada subtarefa é atribuída ao agente ou sub-agente mais qualificado, com base em especialização, custo e restrições de segurança. **Coordenação:** os resultados parciais são agregados, refinados e integrados, por um orquestrador central ou de forma distribuída.

A orquestração pode ser estática (topologia definida em design) ou dinâmica (topologia que emerge por decisões de LLMs em execução). A maioria dos sistemas empresariais adota abordagem híbrida: a estrutura geral é predefinida para consistência e conformidade, mas roteamento, replanejamento e exceções são decididos dinamicamente (AWS, 2025).

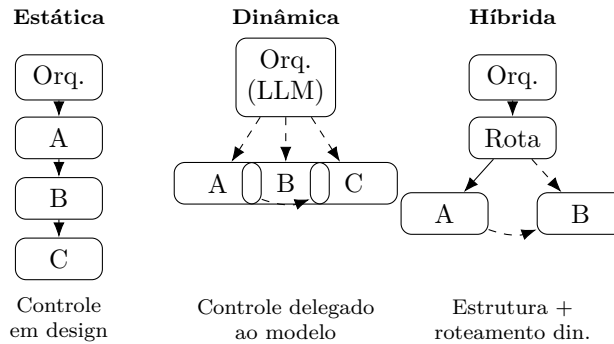


Figura 1: Esquema comparativo das topologias estática, dinâmica e híbrida de orquestração multi-agente, ilustrando o locus da decisão de controle.

3 Padrões de Orquestração em Produção

O ecossistema contemporâneo de engenharia de software voltado à inteligência artificial converge para seis padrões fundamentais de orquestração de agentes (MICROSOFT, 2026; ZYLOS RESEARCH, 2026). A seleção do padrão adequado deve considerar o grau de acoplamento entre as etapas, a necessidade de consenso entre especialistas e a tolerância a latência. A Tabela 1 sintetiza, ao final da seção, os trade-offs de cada padrão.

3.1 Orchestrator-Worker (Supervisor)

O padrão Orchestrator-Worker, frequentemente denominado Supervisor, é a arquitetura mais difundida em sistemas multi-agente comerciais: um orquestrador central recebe a requisição, decompõe o problema em subtarefas, delega cada uma a um agente especialista (*worker*), monitora a execução e sintetiza o resultado final, mantendo o estado global e a consistência do produto (MICROSOFT, 2026). É indicado para workflows cuja decomposição exige julgamento contextual e planos variáveis conforme a entrada, porém adiciona de 15% a 30% de latência e consome 20% a 40% mais tokens devido às chamadas de ida e volta (MICROSOFT, 2026; *dados de fornecedor único — não consolidados por estudo independente*).

O Claude Agent SDK implementa esse padrão via `agent.asTool()` (ANTHROPIC, 2026a), enquanto a Microsoft o mapeia em grafos de fluxo com transições explícitas (MICROSOFT, 2026) — um *trade-off* entre flexibilidade de composição e previsibilidade de execução.

3.2 Sequential Pipeline (Prompt Chaining)

No padrão Sequential Pipeline, os agentes são encadeados em uma ordem linear estrita e predefinida (AWS, 2025). A saída gerada por um agente serve como entrada direta para o agente subsequente, estabelecendo um fluxo determinístico e sequencial (DEVARAJAN, 2026). A força do padrão reside na previsibilidade: o comportamento do sistema é inteiramente determinado pela topologia declarada em design.

Recomenda-se o uso deste padrão em processos com dependências lineares claras e etapas bem delimitadas, tais como ciclos de produção de conteúdo técnico (geração de rascunho → revisão gramatical → verificação de fatos → formatação). Deve ser evitado em workflows que requeiram iterações complexas ou caminhos de retorno (*backtracking*) (LUSHBINARY, 2026).

3.3 Parallel Fan-Out / Fan-In

O padrão Parallel Fan-Out/Fan-In envolve a execução simultânea de múltiplos agentes para processar a mesma tarefa a partir de diferentes especialidades. Os resultados intermediários são consolidados em um nó de agregação (Fan-In) que utiliza votação, média ponderada ou sumarização algorítmica para gerar a saída consolidada (DIGITAL APPLIED, 2026). A agregação exige critérios explícitos de resolução de conflito, uma vez que agentes especializados podem produzir conclusões divergentes.

Este padrão é ideal para análises multi-perspectivas e tomada de decisão crítica sob restrição de tempo, como a análise de investimentos financeiros com agentes em paralelo (PUCEK, 2026).

3.4 Dynamic Handoff (Routing)

No padrão Dynamic Handoff, os agentes decidem autonomamente, durante a execução, quando transferir o controle da conversação para outro agente especialista (OPENAI, 2026). Diferentemente do padrão Supervisor, onde o controle sempre retorna ao orquestrador central, no Handoff a transferência de controle é definitiva ou semi-definitiva, operando um agente por vez (MICROSOFT, 2026). A conversação "viaja" entre especialistas, preservando o histórico acumulado.

Aplica-se com sucesso em centrais de atendimento ao cliente, com um agente de triagem (*router*) transferindo para especialistas em suporte, faturamento ou retenção. O OpenAI Agents SDK trata esta estrutura como cidadã de primeira classe via `handoff()` (OPENAI, 2026), e o Claude Agent SDK a implementa por sub-agentes programáticos em contexto isolado (ANTHROPIC, 2026a).

3.5 Group Chat (Roundtable)

O padrão Group Chat organiza múltiplos agentes em um ambiente de comunicação compartilhado (*thread* de chat). Um agente gerenciador (*chat manager*) ou uma lógica de roteamento em grafo define qual agente deve falar a cada turno, permitindo debate e consenso dinâmico (MICROSOFT,

2026). A riqueza do padrão reside na possibilidade de cruzamento de perspectivas, ao custo de maior latência e de maior dificuldade de determinação do ponto de término da discussão.

É comumente aplicado em dinâmicas de *design thinking*, resolução colaborativa de problemas ou cenários com múltiplos especialistas de negócios que possuem dependências cruzadas (SUBAGENT.DEV, 2026). O padrão Debate, frequentemente agregado ao Group Chat, promove rodadas de argumentação estruturada antes da decisão final (DIGITAL APPLIED, 2026).

3.6 Magentic (Planning-Ledger)

O padrão Magentic é indicado para problemas de escopo aberto e sem fluxo de execução predefinido. O agente principal cria e atualiza dinamicamente um registro de tarefas pendentes e concluídas (*ledger*). Ele avalia o andamento das tarefas em tempo real, gerando novos planos de ação e acionando sub-agentes conforme o diagnóstico situacional evolui (MICROSOFT, 2026). O *ledger* funciona como uma memória de trabalho externa que compensa as limitações da janela de contexto do modelo.

Este padrão é amplamente adotado em Engenharia de Confiabilidade de Sites (SRE) e resposta a incidentes, onde o agente analisa logs, delega diagnósticos e aplica correções à medida que o comportamento do sistema é revelado (ZYLOS RESEARCH, 2026).

Tabela 1. Síntese comparativa dos seis padrões de orquestração quanto a latência, consumo de tokens, adequação a consenso e previsibilidade.

Padrão	Latência	Tokens	Consenso	Previsibilidade	Indicado para
Orchestrator-Worker	Alta	Alta	Média	Alta	Decomposição com julgamento
Sequential	Baixa	Baixa	Baixa	Muito alta	Dependências lineares
Pipeline	Baixa	Alta	Alta	Média	Análise multi-perspectiva
Fan-Out/Fan-In	Média	Média	Baixa	Média	Triagem e especialização
Dynamic Handoff	Alta	Alta	Alta	Baixa	Debate e design colaborativo
Magentic	Alta	Alta	Média	Baixa	Escopo aberto, incidentes

Fonte: elaborado pelo autor com base em Microsoft (2026), OpenAI (2026), Zylos Research (2026), Epsilla (2026) e Digital Applied (2026).

Legenda da Tabela 1: Classificação qualitativa baseada em análise comparativa da literatura técnica revisada. "Latência" e "Tokens": Baixa/Média/Alta referem-se a overhead relativo vs. execução mono-agente equivalente. "Consenso": capacidade inerente ao padrão de agregar opiniões divergentes (Baixa = nenhum mecanismo; Média = agregação algorítmica simples; Alta = debate estruturado/votação). "Previsibilidade": determinismo do fluxo de execução (Muito alta = topologia fixa; Alta = ramificação controlada; Média = roteamento condicional; Baixa = emergente). Critérios derivados de Microsoft (2026) e Zylos Research (2026).

4 Sub-Agents: Padrões de Interação e Compressão de Contexto

Além dos padrões de orquestração de alto nível, a operação eficiente de sub-agentes em produção depende de três categorias operacionais de interação (EPSILLA, 2026):

Synchronous "Wait and See": o agente pai suspende sua execução e aguarda o retorno do sub-agente, comportando-se de forma análoga a uma chamada de função complexa. É o modo mais simples de raciocinar e depurar, porém limita o paralelismo do sistema.

Asynchronous "Fire and Forget": o agente pai delega uma tarefa e continua seu fluxo

de execução em paralelo, útil para processamento assíncrono ou disparos de eventos. Exige mecanismos de correlação para consumir o resultado posteriormente. **Scheduled "Future Execution"**: o sub-agente é programado para executar em um momento futuro predeterminado ou sob gatilhos temporais específicos. É a base da automação orientada a eventos e da execução recorrente.

O insight central desse desacoplamento não reside apenas no processamento paralelo, mas na compressão do contexto de inferência. Ao isolar sub-tarefas em sub-agentes dotados de escopos de variáveis e ferramentas restritas, reduz-se o volume total de tokens acumulados nas janelas de contexto dos modelos principais, prevenindo a degradação da acurácia causada pelo fenômeno de "esquecimento no meio do contexto" (*lost in the middle*) (EPSILLA, 2026; VELLUM AI, 2025). O agente pai recebe apenas o resultado sintetizado da investigação, e não o rastro completo das ferramentas acionadas — com redução reportada de até 90% no consumo de tokens do agente pai em configurações específicas (EPSILLA, 2026; *dado de fornecedor único — não verificado independentemente*).

A compressão de contexto opera, portanto, em duas dimensões: (i) *temporal*, ao descartar ou resumir histórico obsoleto; e (ii) *espacial*, ao delimitar o escopo de ferramentas e variáveis de cada sub-agente. A combinação das duas é o que torna viável a execução de tarefas de longa duração sem estouro da janela de contexto do modelo.

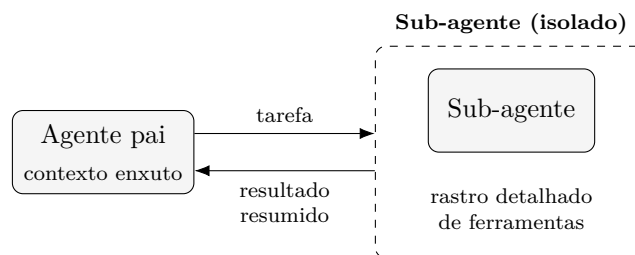


Figura 2: Representação do fluxo de compressão de contexto: o rastro detalhado das ferramentas permanece no sub-agente, enquanto o agente pai recebe apenas o resultado resumido.

5 Ecossistema de Frameworks e Ferramentas

O mercado de ferramentas para orquestração de agentes registrou rápida consolidação em torno de frameworks especializados (GHEWARE, 2026; JETTHOUGHTS, 2026). A escolha do framework não é meramente técnica: ela determina o modelo mental dos desenvolvedores, os limites de segurança disponíveis e a facilidade de observabilidade. A seguir, analisam-se os seis principais frameworks do ecossistema.

5.1 LangGraph

Extensão do ecossistema LangChain para fluxos cíclicos modelados como grafos de estado, com controle preciso de execução e gerenciamento de estados (AGENT MARKET CAP, 2026). Diferenciais: *checkpointing* nativo, *time-travel debugging* e integração LangSmith; limitação: curva de aprendizado íngreme (OPENAGENTS, 2026).

5.2 CrewAI

Adota a metáfora organizacional de "*crews*" — agentes com papéis, ferramentas e memórias predefinidas executando tarefas hierárquicas ou sequenciais (META INTELLIGENCE, 2025). Protolpagem rápida, porém com controle menos granular sobre fluxos dinâmicos complexos (GHEWARE, 2026).

5.3 OpenAI Agents SDK

Foca em ‘handoffs’ simplificados de controle de conversação, com agente como primitiva fundamental, *guardrails* declarativos e *tracing* integrado (OPENAI, 2026), favorecendo centrais de atendimento e fluxos de especialização.

5.4 Claude Agent SDK

Foca no Model Context Protocol (MCP) e na execução integrada a ambientes de desenvolvimento, com o mesmo *harness* do Claude Code, sub-agentes em contexto isolado e paralelização nativa (ANTHROPIC, 2026b; WINBUZZER, 2026) — o isolamento de contexto é a base de sua proposta de escala.

5.5 Microsoft Agent Framework

Sucessor corporativo do AutoGen, integrado ao Azure, com ênfase em segurança e conformidade via *workflow graphs* de transições explícitas (ROCKB, 2026), posicionado para setores regulados.

5.6 Spring AI Agent Utils

Voltado ao ecossistema Java/Spring Boot, oferece isolamento de sub-agentes por *Task tools*, *multi-model routing* e suporte ao protocolo A2A (TZOLOV, 2026), sendo a via natural para bases de código Java consolidadas.

5.7 Matriz de Compatibilidade de Frameworks

A Tabela 2 apresenta a avaliação de compatibilidade nativa entre os frameworks analisados e os padrões de orquestração discutidos, consolidada a partir da literatura técnica revisada.

Tabela 2. Matriz de compatibilidade entre frameworks e padrões de orquestração

Framework	Orchestra- tor	Sequential	Parallel	Handoff	Group Chat	A2A	MCP
Lang- Graph CrewAI	Nativo	Nativo	Nativo	Via nós	Via <i>custom</i>	—	Via
	Hierárquico	Nativo	Nativo	Limitado	Não suportado	Sim	LangChain Comunitário
OpenAI Agents SDK	Agent-as- Tool	Manual	Via <i>handoffs</i>	Nativo	Não suportado	—	Sim
Claude Agent SDK	Agent tool	Programá- tico	Sub-agentes	Sub-agentes	Agent Teams	Planejado	Nativo
MS Agent Framework	<i>Workflow graphs</i>	<i>Workflow graphs</i>	Nativo	Nativo	<i>GroupChat</i> legado	Sim	<i>Built-in</i>
Spring AI Agent Utils	<i>Task tool</i>	Programá- tico	<i>Task tool</i>	<i>Task tool</i>	Não suportado	Sim	Sim

Fonte: adaptado de Microsoft (2026), OpenAI (2026), Anthropic (2026a, 2026b), Tzolov (2026), RockB (2026) e Zylos Research (2026).

Legenda da Tabela 2: Avaliação de compatibilidade nativa (sem *workarounds* ou extensões comunitárias). "Nativo"= suporte direto na API principal do framework; "Hierárquico"= via padrão Orchestrator-Worker interno; "Via nós"= implementável compondo nós de grafo; "Via *handoffs*"= usando primitiva de transferência de controle; "Programático"= via código personalizado do desenvolvedor; "*Custom*"= exige implementação *ad hoc*; "Não suportado"= sem mecanismo documentado;

”Planejado”= no *roadmap* público; ”Comunitário”= via extensão de terceiros; ”Legado”= herdado de versão anterior (AutoGen); ”*Built-in*”= integrado ao *runtime*. Critério: documentação oficial + repositórios de exemplos (acesso jun/2026).

5.8 Critérios de Seleção de Framework

A Tabela 3 propõe um critério estruturado de seleção, útil para arquitetos que enfrentam a decisão de adoção. Cada critério deve ser ponderado pelos requisitos do projeto.

Tabela 3. Matriz de critérios para seleção de framework

Critério	Peso sugerido	Lang-Graph	CrewAI	OpenAI SDK	Claude SDK	MS AF	Spring AI
Controle de estado	Alto	●●●	●●○	●●○	●●○	●●●	●●○
Ergonomia/Prototipação	Médio	●●○	●●●	●●○	●●○	●●○	●●○
Segurança/Conformidade	Alto	●●○	●●○	●●○	●●○	●●●	●●○
Observabilidade	Alto	●●●	●●○	●●●	●●○	●●○	●●○
Ecossistema/Integração	Médio	●●●	●●○	●●○	●●○	●●●	●●●

Legenda: ●●● forte; ●●○ moderado; ●○● limitado. Fonte: elaborado pelo autor com base em Agent Market Cap (2026), OpenAgents (2026), Gheware (2026), RockB (2026) e Shakudo (2026).

Legenda da Tabela 3: Atribuição de escores por critério baseada em evidência documental (documentação oficial, *benchmarks* publicados, *white papers* dos fornecedores). ”Forte (●●●)”: suporte nativo e maduro, documentado com exemplos de produção; ”Moderado (●●○)”: suporte parcial, exige configuração adicional ou apresenta limitações conhecidas; ”Limitado (●○●)”: suporte incipiente, *workaround* necessário ou ausente. Pesos sugeridos: Alto = critério bloqueante para adoção em setores regulados/críticos; Médio = diferencial competitivo. Avaliação realizada jun/2026; sujeita a atualização conforme *releases* dos frameworks.

6 Aplicações Setoriais e Estudos de Caso

A adoção de *dynamic workflows* não é meramente teórica: casos de produção documentados ilustram a aplicabilidade dos padrões apresentados em domínios diversos, permitindo extrair princípios de design transferíveis. A correspondência entre padrão e setor revela uma lógica recorrente — a topologia é escolhida em função da necessidade de determinismo, paralelismo ou replanejamento contínuo.

No setor jurídico, um escritório de advocacia utiliza um *pipeline* sequencial de quatro agentes para geração de contratos (seleção de *template*, customização de cláusulas, verificação de *compliance* e avaliação de risco), refletindo a natureza regulatória e a necessidade de determinismo (SHAKUDO, 2025). No setor financeiro, a análise de investimentos emprega o padrão Parallel Fan-Out/Fan-In, com agentes executando em paralelo análises fundamentalistas, técnicas, de sentimento e ESG, convergindo para um relatório consolidado (PUCEK, 2026). No atendimento ao cliente, o padrão Dynamic Handoff aplica um agente de triagem que transfere o controle para especialistas em suporte, faturamento ou retenção, otimizando o uso de contexto (OPENAI, 2026). Na engenharia de confiabilidade (SRE), o padrão Magentic sustenta a resposta a incidentes, onde a imprevisibilidade do domínio inviabiliza topologia estática (ZYLOS RESEARCH, 2026). Em tarefas de produção de conteúdo técnico, o Orchestrator-Worker coordena rascunho, revisão, verificação de fatos e formatação (VELLUM AI, 2025). A Figura 3 mapeia esse alinhamento.

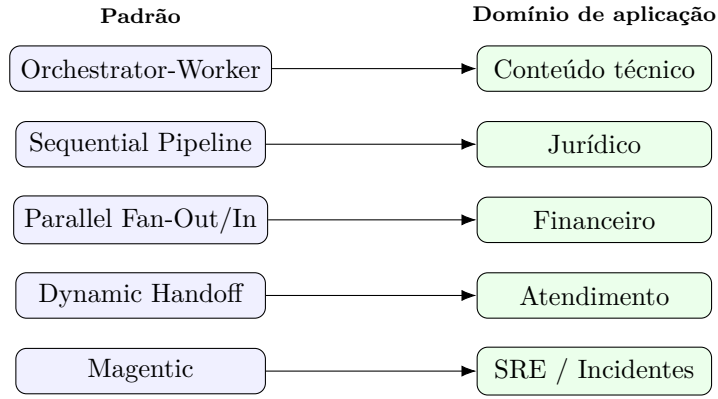


Figura 3: Mapa de alinhamento entre padrões de orquestração e domínios de aplicação setorial.

7 Modelagem de Custos e Latência

A viabilidade econômica de sistemas multi-agente depende de uma modelagem cuidadosa de custo e latência. O overhead de latência do Orchestrator-Worker situa-se entre 15% e 30%, e o de tokens entre 20% e 40%, atribuíveis às chamadas de ida e volta entre orquestrador e *workers* (MICROSOFT, 2026; *dados de fornecedor único — não consolidados por estudo independente*). Tais números devem ser confrontados com os ganhos de qualidade e de compressão de contexto antes da adoção do padrão.

Em contrapartida, a compressão de contexto por sub-agentes reduz o consumo de tokens do agente pai em até 90% em configurações reportadas (EPSILLA, 2026). Para tarefas de longa duração, esse ganho tende a superar o overhead de orquestração, tornando o sistema multi-agente mais barato no total do que a alternativa mono-agente com contexto inflado. A arquitetura de custos deve seguir o roteamento dinâmico baseado em complexidade (*difficulty-aware routing*): modelos menores para roteamento, extração e verificações; modelos avançados reservados a planejamento e consolidação (GROKIPEDIA, 2026). A Tabela 4 ilustra o efeito combinado das decisões de arquitetura.

Tabela 4. Efeito qualitativo de decisões de arquitetura sobre custo e latência

Decisão de arquitetura	Efeito em custo	Efeito em latência	Fonte
Sub-agentes com compressão de contexto	Reduz	Neutro/Positivo	Epsilla (2026)
Orchestrator-Worker	Aumenta	Aumenta	Microsoft (2026)
Parallel Fan-Out	Aumenta	Reduz	Digital Applied (2026); Pucek (2026)
Difficulty-aware routing	Reduz	Neutro	Grokikipedia (2026)
Checkpointing/pruning a 70%	Reduz	Neutro	Stackviv (2026)

Fonte: elaborado pelo autor com base nas referências citadas.

Nota da Tabela 4: Valores qualitativos baseados em fontes primárias dos respectivos fornecedores/frameworks (Microsoft, Epsilla, Digital Applied, Grokikipedia, Stackviv). Não constituem meta-análise ou estudo independente consolidado; servem como indicativo direcional para decisões de arquitetura.

Recomenda-se aplicar compactação (sumarização e *pruning*) quando o contexto acumulado excede 70% do limite do modelo (STACKVIV, 2026), antecipando a degradação de qualidade antes que ela se torne perceptível.

8 Diretrizes de Engenharia para Produção

A transição de protótipos de agentes para sistemas multi-agente de produção estável exige a aplicação de rigorosas diretrizes de engenharia (WITNESSAI, 2026; STACKVIV, 2026). As subseções a seguir detalham os cinco eixos centrais.

8.1 Gerenciamento Dinâmico de Contexto

À medida que múltiplos agentes interagem, a janela de contexto pode saturar rapidamente. Sistemas robustos em produção devem implementar sumarização incremental e *pruning* de histórico — remoção de metadados de execução irrelevantes e respostas intermediárias de ferramentas das mensagens enviadas ao modelo principal. Recomenda-se aplicar compactação quando o contexto acumulado excede 70% do limite do modelo (STACKVIV, 2026). O *pruning* preserva os artefatos de decisão (delegações, resultados) e descarta o rastro de ferramentas de baixo valor.

8.2 Otimização de Custos e Latência

Conforme modelado na Seção 7, recomenda-se o *difficulty-aware routing* (GROKIPEDIA, 2026). Complementarmente, o *caching* de contexto e a reutilização de sub-resultados reduzem o custo marginal de requisições repetitivas.

8.3 Segurança e Isolamento

Os sub-agentes devem operar sob o princípio do menor privilégio, restringindo o acesso a APIs externas e bancos de dados ao escopo exato da tarefa delegada. Mecanismos de *circuit breaker* e cotas rígidas por agente previnem loops de execução infinitos (WITNESSAI, 2026). O isolamento de contexto — garantido por frameworks como o Claude Agent SDK (ANTHROPIC, 2026b) — limita o raio de explosão de um comprometimento de um sub-agente.

8.4 Human-in-the-Loop

Diversos padrões suportam participação humana: observadores em *group chat*, revisores em ciclos *maker-checker* e escalção em *handoff*. Pontos de verificação obrigatórios tornam a orquestração síncrona, e o estado deve ser persistido nos *checkpoints* para retomada sem *replay* (MICROSOFT LEARN, 2026). A presença humana em pontos críticos de decisão é particularmente recomendada para domínios de alto risco, onde a delegação automática plena ainda apresenta lacunas de segurança.

8.5 Observabilidade

O *stack* mínimo de observabilidade combina LangSmith (rastreamento de agentes), OpenTelemetry (métricas de infraestrutura) e *dashboards* de *workflow engine*, instrumentando toda operação e todo *handoff*, com rastreamento de latência, consumo de recursos e qualidade de saída por agente (SHAKUDO, 2026; APISCOUT, 2026). Sem observabilidade, a depuração de topologias dinâmicas torna-se inviável, dado o não determinismo das decisões de roteamento.

9 Riscos, Limitações e Estratégias de Mitigação

Apesar dos ganhos, a adoção de *dynamic workflows* introduz riscos que devem ser explicitamente geridos. Esta seção cataloga as limitações documentadas e propõe mitigações alinhadas às diretrizes da Seção 8.

Não determinismo de roteamento. Decisões de LLMs em tempo de execução podem divergir entre execuções equivalentes. *Mitigação:* abordagem híbrida com estrutura predefinida e limites de transição (AWS, 2025), persistindo traços para auditoria (SHAKUDO, 2026; APISCOUT, 2026).

Custo de tokens da orquestração. O overhead de 20% a 40% em tokens e 15% a 30% em latência do Orchestrator-Worker (MICROSOFT, 2026; *dados de fornecedor único — não consolidados por estudo independente*) pode inviabilizar alto volume. *Mitigação:* *difficulty-aware routing* (GROKIPEDIA, 2026) e compressão de contexto (EPSILLA, 2026).

Depuração de topologias dinâmicas. A multiplicidade de caminhos dificulta a causa raiz de falhas. *Mitigação:* observabilidade integral (SHAKUDO, 2026; APISCOUT, 2026) e *time-travel debugging* com *checkpointing* (AGENT MARKET CAP, 2026).

Riscos de segurança e loops. Privilégios excessivos e ausência de *circuit breaker* induzem loops ou exfiltração. *Mitigação:* menor privilégio e cotas rígidas (WITNESSAI, 2026).

Padronização de interoperabilidade. A fragmentação de protocolos (A2A, MCP) limita a portabilidade. *Mitigação:* frameworks com suporte nativo aos protocolos padronizados (TZOLOV, 2026; MICROSOFT, 2026).

10 Tendências Emergentes

Seis tendências moldam o futuro dos *dynamic workflows* (ZYLOS RESEARCH, 2026; MICROSOFT LEARN, 2026):

DAGs (Directed Acyclic Graphs): workflows modelados como grafos acíclicos dirigidos, com LangGraph como expoente. **Orquestração *Event-Driven*:** agentes reagem a eventos em barramentos (Kafka, RabbitMQ). **Modelo *Actor*:** cada agente como ator com caixa postal, popularizado por AutoGen e MAF. **Roteamento Dinâmico por Dificuldade:** classificadores estimam a complexidade da *query* e alocam recursos proporcionalmente. **Orquestração Federada:** coordenação entre *cloud* e *edge* através de fronteiras de confiança, com o protocolo A2A como bloco fundamental. **Workflows Auto-Otimizáveis:** meta-agentes ajustam a topologia do *workflow* com base em feedback de execuções anteriores.

Essas tendências apontam para a convergência de dois movimentos: a formalização de grafos como linguagem comum de especificação (itens 1 a 3) e a autonomização progressiva da topologia (itens 4 a 6). A adoção antecipada de protocolos de interoperabilidade (A2A, MCP) é condição para capturar os benefícios da orquestração federada e auto-otimizável sem *lock-in* de fornecedor.

11 Roadmap de Adoção em Produção

A partir das diretrizes e riscos discutidos, propõe-se um roteiro de adoção em três fases, desenhado para minimizar risco enquanto se constrói competência organizacional.

Fase 1 — Fundação (Semanas 1–4). Estabelecer o *stack* de observabilidade (LangSmith, OpenTelemetry) e o modelo de isolamento de contexto e privilégios (WITNESSAI, 2026; SHAKUDO, 2026), iniciando com agentes generalistas simples.

Fase 2 — Orquestração Híbrida (Semanas 5–10). Introduzir Orchestrator-Worker ou Sequential Pipeline conforme a tarefa, com compressão de contexto por sub-agentes (EPSILLA, 2026), *difficulty-aware routing* (GROKIPEDIA, 2026) e *checkpoints* para retomada sem *replay* (MICROSOFT LEARN, 2026).

Fase 3 — Escala e Autonomia (Semanas 11+). Expandir para Parallel Fan-Out/Fan-In e Dynamic Handoff conforme a necessidade de análise multi-perspectiva, habilitar *human-in-the-loop* e considerar Magentic para domínios de escopo aberto (ZYLOS RESEARCH, 2026), reavaliando a topologia à luz das métricas de custo e latência (Tabela 4).

A recomendação central é evoluir de topologias estáticas para dinâmicas de forma incremental, sustentada por observabilidade desde o primeiro dia e por uma arquitetura em camadas que combine

durabilidade, raciocínio de agente, coordenação inter-serviço e monitoramento.

12 Conclusão

Os padrões de orquestração multi-agente evoluíram de conceitos acadêmicos para pilares de arquitetura de software em produção empresarial. A escolha do padrão e do framework corretos deve ser guiada por requisitos pragmáticos de latência, custos e complexidade da tarefa, e não por preferências de implementação. A compressão de contexto por meio de sub-agentes consolida-se como o principal benefício prático, com reduções documentadas de até 90% no consumo de tokens do agente pai em configurações reportadas (EPSILLA, 2026; *dado de fornecedor único — não verificado independentemente*).

A engenharia de sistemas de produção converge para o princípio de iniciar com agentes generalistas simples e aplicar especialização em sub-agentes apenas quando houver evidências mensuráveis de ganhos de performance ou de conformidade. Recomenda-se uma arquitetura híbrida em camadas — durabilidade (Temporal), raciocínio de agente (LangGraph), coordenação inter-serviço (Kafka + A2A) e observabilidade desde o primeiro dia — evoluindo de topologias estáticas para dinâmicas de forma incremental. Limitações atuais incluem a depuração de topologias dinâmicas, o custo de tokens da orquestração e a necessidade de padronização de protocolos (A2A, MCP) entre frameworks.

Perspectivas futuras apontam para a consolidação de grafos (DAGs) como linguagem de especificação, para a orquestração orientada a eventos e para workflows auto-otimizáveis mediados por meta-agentes. O sucesso da adoção dependerá da disciplina de engenharia aplicada à observabilidade, à segurança por menor privilégio e ao *human-in-the-loop* em pontos de decisão de alto risco.

Agradecimentos

[A preencher pelo autor — registrar agradecimentos a orientadores, pares revisores e instituições de fomento; indicar códigos de financiamento (bolsa, edital ou projeto), se aplicável. Omitir esta seção caso não haja contrapartidas a registrar.]

Conflito de Interesses

Os autores declaram não haver conflitos de interesses financeiros, comerciais ou acadêmicos relacionados ao presente trabalho. A pesquisa foi conduzida de forma independente, sem financiamento de fornecedores de frameworks ou plataformas citados. As análises de frameworks baseiam-se em documentação pública, literatura técnica e *benchmarks* de terceiros, sem participação em programas de *early access* ou parcerias comerciais que pudessem comprometer a imparcialidade da avaliação.

Referências

AGENT MARKET CAP. LangGraph vs AutoGen vs CrewAI vs DSPy: The 2026 Multi-Agent Framework Decision Guide. **Agent Market Cap**, 2026. Disponível em: <https://agentmarketcap.ai/blog/2026/04/11/langgraph-autogen-crewai-dspy-multi-agent-orchestration-2026>. Acesso em: 12 jul. 2026.

ANTHROPIC. **Claude Agent SDK**: Subagents in the SDK. **Anthropic Documentation**, 2026a. Disponível em: <https://code.claude.com/docs/en/agent-sdk/subagents>. Acesso em: 12 jul. 2026.

ANTHROPIC. **Claude Code**: Orchestrate subagents at scale with dynamic workflows. **Anthropic Documentation**, 2026b. Disponível em: <https://code.claude.com/docs/en/workflows>. Acesso em: 12 jul. 2026.

APISCOUT. **OpenAI Agents SDK: Architecture Patterns 2026**. APIScout, 2026. Disponível em: <https://apiscout.dev/guides/openai-agents-sdk-architecture-patterns-2026>. Acesso em: 12 jul. 2026.

AWS. **Agentic AI patterns and workflows on AWS**. Amazon Web Services, 2025. Disponível em: <https://docs.aws.amazon.com/prescriptive-guidance/latest/agentic-ai-patterns/introduction.html>. Acesso em: 12 jul. 2026.

DEVARAJAN, Vishalini. **Common Workflow Patterns for AI Agents**. GUVI Blog, 2026. Disponível em: <https://www.guvi.in/blog/common-workflow-patterns-for-ai-agents>. Acesso em: 12 jul. 2026.

DIGITAL APPLIED. **Multi-Agent Orchestration: 5 Patterns That Work in 2026**. Digital Applied, 2026. Disponível em: <https://www.digitalapplied.com/blog/multi-agent-orchestration-5-patterns-that-work>. Acesso em: 12 jul. 2026.

EPSILLA. **The 3 Essential Sub-Agent Patterns for Production-Grade AI Systems**. Epsilla Blog, 2026. Disponível em: <https://www.epsilla.com/blogs/2026-03-14-ai-sub-agent-patterns>. Acesso em: 12 jul. 2026.

GARTNER. Gartner Predicts 40% of Enterprise Apps Will Feature Task-Specific AI Agents by 2026, Up from Less Than 5% in 2025. **Gartner Newsroom**, 26 ago. 2025. Disponível em: <https://www.gartner.com/en/newsroom/press-releases/2025-08-26-gartner-predicts-40-percent-of-enterprise-apps-will-feature-task-specific-ai-agents-by-2026-up-from-less-than-5-percent-in-2025>. Acesso em: 12 jul. 2026.

GHEWARE, Rajesh. **LangGraph vs CrewAI vs AutoGen: Complete AI Agent Framework Comparison 2026**. DevOps Gheware, 2026. Disponível em: <https://devops.gheware.com/blog/posts/langgraph-vs-crewai-vs-autogen-comparison-2026.html>. Acesso em: 12 jul. 2026.

GROKIPEDIA. **AI Agent Orchestration**. Grokipedia, 2026. Disponível em: https://grokipedia.com/page/AI_Agent_Orchestration. Acesso em: 12 jul. 2026.

JETTHOUGHTS. **LangGraph vs CrewAI vs AutoGen: Open Source Alternatives 2025**. JetThoughts, 2026. Disponível em: <https://jetthoughts.com/blog/autogen-crewai-langgraph-ai-agent-frameworks-2025>. Acesso em: 12 jul. 2026.

LUSHBINARY. **Multi-Agent AI Orchestration Patterns: Supervisor, Swarm, Pipeline & Router**. Lushbinary, 2026. Disponível em: <https://lushbinary.com/blog/multi-agent-orchestration-patterns-supervisor-swarm-pipeline-router-guide>. Acesso em: 12 jul. 2026.

META INTELLIGENCE. **LangGraph vs CrewAI vs AutoGen: AI Agent Framework Comparison [2026]**. Meta Intelligence, 2025. Disponível em: <https://www.meta-intelligence.tech/en/insight-ai-agent-frameworks>. Acesso em: 12 jul. 2026.

MICROSOFT. **AI Agent Orchestration Patterns**. Microsoft Learn, 2026. Disponível em: <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/ai-agent-design-patterns>. Acesso em: 12 jul. 2026.

MICROSOFT LEARN. **Multi-agent patterns**. Microsoft Learn, 2026. Disponível em: <https://learn.microsoft.com/en-us/agents/architecture/multi-agent-patterns>. Acesso em: 12 jul. 2026.

OPENAI. **Agents SDK: Orchestration and handoffs**. OpenAI Documentation, 2026. Disponível em: <https://developers.openai.com/api/docs/guides/agents/orchestration>. Acesso em: 12 jul. 2026.

OPENAGENTS. **CrewAI vs LangGraph vs AutoGen vs OpenAgents: Best AI Agent Framework**. OpenAgents, 2026. Disponível em: <https://openagents.org/blog/posts/2026-02-23-open-source-ai-agent-frameworks-compared>. Acesso em: 12 jul. 2026.

PUCEK, Bartek. **Orchestration Patterns for AI Agent Workflows**. The Thinking Company, 2026. Disponível em: <https://thinking.inc/en/blue-ocean/agentic/agent-orchestration-patterns>. Acesso em: 12 jul. 2026.

ROCKB. **Microsoft Agent Framework 2026: AutoGen Successor Explained**. RockB, 2026. Disponível em: <https://baeseokjae.github.io/posts/microsoft-agent-framework-2026/>. Acesso em: 12 jul. 2026.

SHAKUDO. **Enterprise Agentic AI Workflow Patterns for 2025**. Shakudo, 2025. Disponível em: https://cdn.prod.website-files.com/625447c67b621ab49bb7e3e5/69388ca4cdb5836ee83b10f5_69388ca257d8a9675e92aeb8_agentic-ai-workflow-patterns-whitepaper.pdf. Acesso em: 12 jul. 2026.

SHAKUDO. **Multi-Agent Systems Transform Enterprise AI in 2026**. Shakudo, 2026. Disponível em: https://cdn.prod.website-files.com/625447c67b621ab49bb7e3e5/696fdd9dcee4a9c14251480b_696fdd9c3876035a277d2a6f_multi-agent-systems-2026-trends-whitepaper.pdf. Acesso em: 12 jul. 2026.

STACKVIV. **Agentic AI & Multi-Agent Systems: Advanced Guide**. Stackviv, 2026. Disponível em: <https://stackviv.ai/blog/agentic-ai-multi-agent-systems-guide>. Acesso em: 12 jul. 2026.

SUBAGENT.DEV. **Orchestration Patterns**. SubAgent.Dev, 2026. Disponível em: <https://subagent.developersdigest.tech/patterns>. Acesso em: 12 jul. 2026.

TZOLOV, Christian. **Spring AI Agentic Patterns (Part 4): Subagent Orchestration**. Spring Blog, 2026. Disponível em: <https://spring.io/blog/2026/01/27/spring-ai-agentic-patterns-4-task-subagents/>. Acesso em: 12 jul. 2026.

VELLUM AI. **The 2026 Guide to AI Agent Workflows**. Vellum AI, 2025. Disponível em: <https://www.vellum.ai/blog/agentic-workflows-emerging-architectures-and-design-patterns>. Acesso em: 12 jul. 2026.

WINBUZZER. **Anthropic Shows How to Scale Claude Code with Subagents and MCP**. WinBuzzer, 2026. Disponível em: <https://winbuzzer.com/2026/03/24/anthropic-claude-code-subagent-mcp-advanced-patterns-xcxwn>. Acesso em: 12 jul. 2026.

WITNESSAI. **AI Agent Orchestration: Managing Multi-Agent Workflows**. WitnessAI, 2026. Disponível em: <https://witness.ai/blog/ai-agent-orchestration>. Acesso em: 12 jul. 2026.

ZYLOS RESEARCH. **Agent Workflow Orchestration Patterns: DAG, Event-Driven, and Actor Models**. Zylos Research, 2026. Disponível em: <https://zylos.ai/research/2026-04-14-agent-workflow-orchestration-patterns>. Acesso em: 12 jul. 2026.